



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РФ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«ЛИПЕЦКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Институт
Кафедра

Компьютерных наук
Автоматизированных систем управления

ЛАБОРАТОРНАЯ РАБОТА №4

По дисциплине: «Технологии разработки программного обеспечения».
На тему: «Управление версиями и доработка прототипа с использованием Git Flow. Unit-тесты».

Студент

ПИ-23-1

группа

подпись, дата

Петров М. В.

фамилия, инициалы

Руководитель

ученая степень, ученое звание

подпись, дата

Патрин Д. Р.

фамилия, инициалы

1. Цель работы

Освоение методологии Git Flow для управления разработкой программного обеспечения, получение практических навыков работы с ветками (main, develop, feature-ветки), слиянием (merge) и разрешением потенциальных merge-конфликтов, а также написание unit-тестов для доработанного функционала.

2. Выбранная предметная область

Система уведомлений об акциях и скидках — распределённая backend-система для автоматической рассылки персонализированных SMS-уведомлений покупателям. Реализована на C++17 с использованием фреймворка Crow, базы данных SQLite и интеграции с API smsaero.ru.

По итогам ЛР №2 и ЛР №3 в системе реализованы: создание кампании, отправка SMS с персонализацией по шаблону {name}, отписка номера. В рамках данной работы система расширена двумя фичами из нереализованных use case ЛР №2: аналитика по кампании и повторная отправка упавших кампаний.

3. Структура репозитория перед доработками

Перед началом работы репозиторий содержал коммиты начального прототипа из ЛР №2–3. Точкой отсчёта для данной лабораторной работы является коммит 26c901c — «feat: initial prototype from LR2-LR3», помеченный веткой main. Команда для просмотра:

```
git log --oneline --graph --all
```

На рисунке 1 представлена полная история репозитория. Коммиты ниже метки main (26c901c) — это исходное состояние до доработок ЛР №4.

```
whocares@DESKTOP-BT68DGB:~/promo-notify-cpp$ git log --oneline --graph --all
* 3595029 (HEAD -> develop) Merge branch 'feature/unit-tests' into develop
|
| * ec0daeb chore: update build config and dockerfile for gtest
| * 6c366f4 test: add unit tests for personalize and resend logic
|/
* 77ae173 feat: add resend endpoint POST /campaigns/<id>/resend
* 05ff647 feat: add campaign stats endpoint GET /campaigns/<id>/stats
* 26c901c (main) feat: initial prototype from LR2-LR3
* 7af3382 (origin/main, origin/HEAD) sms delivered
* 22de415 first launch
* e581aad CMake: Lists implemented
* 958765b chore: git ignore added
* 6991c4f chore: init project structure
* ca2cb97 first commit
```

Рисунок 1 — История репозитория (базовая точка — коммит 26c901c, ветка main)

4. Список выбранных фич

Выбраны два нереализованных use case из таблицы границ мини-прототипа (ЛР №2): аналитика и отчётность, а также повышение надёжности доставки (retry-логика).

№	Название фичи	Ветка	Статус
1	Аналитика кампании GET /campaigns/<id>/stats	feature/campaign-stats	В ожидании тестирования (покрыто unit-тестами)
2	Повторная отправка POST /campaigns/<id>/resend	feature/campaign-resend	В ожидании тестирования (покрыто unit-тестами)

Фича 1 реализует маршрут GET /campaigns/<id>/stats, возвращающий полную информацию о кампании: название, номер телефона, шаблон сообщения, имя получателя и текущий статус.

Фича 2 реализует маршрут POST /campaigns/<id>/resend — повторную отправку SMS для кампаний со статусом failed. Перед повторной отправкой проверяется статус кампании и наличие номера в списке отписавшихся.

5. Список веток и коммитов

Разработка велась по методологии Git Flow. Для каждой фичи создавалась отдельная feature-ветка от develop, после завершения работы ветка сливалась обратно командой `git flow feature finish`.

Последовательность команд:

```
# Инициализация Git Flow
git flow init

# Фича 1 – аналитика кампании
git flow feature start campaign-stats
git commit -m "feat: add campaign stats endpoint GET
/campaigns/<id>/stats"
git flow feature finish campaign-stats

# Фича 2 – повторная отправка
git flow feature start campaign-resend
git commit -m "feat: add resend endpoint POST /campaigns/<id>/resend"
git flow feature finish campaign-resend

# Unit-тесты
git flow feature start unit-tests
git commit -m "test: add unit tests for personalize and resend logic"
git flow feature finish unit-tests
```

Итоговый список веток и коммитов после завершения всех фич представлен на рисунке 2. Видно что все feature-ветки слиты в develop, ветка main указывает на исходный прототип.

```

● whocares@DESKTOP-BT68DGB:~/promo-notify-cpp$ git branch -a
git log --oneline --all
* develop
  main
  remotes/origin/HEAD -> origin/main
  remotes/origin/main
3595029 (HEAD -> develop) Merge branch 'feature/unit-tests' into develop
ec0daeb chore: update build config and dockerfile for gtest
6c366f4 test: add unit tests for personalize and resend logic
77ae173 feat: add resend endpoint POST /campaigns/<id>/resend
05ff647 feat: add campaign stats endpoint GET /campaigns/<id>/stats
26c901c (main) feat: initial prototype from LR2-LR3
7af3382 (origin/main, origin/HEAD) sms delivered
22de415 first launch
e581aad CMake: Lists implemented
958765b chore: git ignore added
6991c4f chore: init project structure
ca2cb97 first commit

```

Рисунок 2 — Финальный список веток и коммитов репозитория

6. Unit-тесты

Для покрытия нового функционала написаны unit-тесты с использованием фреймворка Google Test. Тесты расположены в файле `app/tests/test_campaign_service.cpp` и разбиты на два набора.

6.1. PersonalizeTest — тесты персонализации

Проверяют корректность подстановки имени получателя в шаблон сообщения:

```

TEST(PersonalizeTest, ReplacesNamePlaceholder) {
    std::string result = personalize(
        "Привет, {name}! Скидка 20%", "Михаил");
    EXPECT_EQ(result, "Привет, Михаил! Скидка 20%");
}

TEST(PersonalizeTest, NoPlaceholderUnchanged) {
    std::string result = personalize("Скидка 20% на всё", "Михаил");
    EXPECT_EQ(result, "Скидка 20% на всё");
}

TEST(PersonalizeTest, EmptyName) {
    std::string result = personalize("Привет, {name}!", "");
    EXPECT_EQ(result, "Привет, !");
}

```

6.2. ResendTest — тесты логики повторной отправки

Проверяют что повторная отправка разрешена только для кампаний со статусом failed:

```
TEST(ResendTest, AllowsResendForFailed) {  
    EXPECT_TRUE(canResend("failed"));  
}  
  
TEST(ResendTest, DeniesResendForSent) {  
    EXPECT_FALSE(canResend("sent"));  
}  
  
TEST(ResendTest, DeniesResendForCreated) {  
    EXPECT_FALSE(canResend("created"));  
}  
  
TEST(ResendTest, DeniesResendForSkipped) {  
    EXPECT_FALSE(canResend("skipped"));  
}
```

Запуск тестов в Docker-контейнере:

```
docker compose exec app /app/promo_tests
```

Результат запуска представлен на рисунке 3. Все 7 тестов из 2 наборов прошли успешно (PASSED).

```

[=====] Running 7 tests from 2 test suites.
[-----] Global test environment set-up.
[-----] 3 tests from PersonalizeTest
[ RUN    ] PersonalizeTest.ReplacesNamePlaceholder
[      OK ] PersonalizeTest.ReplacesNamePlaceholder (0 ms)
[ RUN    ] PersonalizeTest.NoPlaceholderUnchanged
[      OK ] PersonalizeTest.NoPlaceholderUnchanged (0 ms)
[ RUN    ] PersonalizeTest.EmptyName
[      OK ] PersonalizeTest.EmptyName (0 ms)
[-----] 3 tests from PersonalizeTest (0 ms total)

[-----] 4 tests from ResendTest
[ RUN    ] ResendTest.AllowsResendForFailed
[      OK ] ResendTest.AllowsResendForFailed (0 ms)
[ RUN    ] ResendTest.DeniesResendForSent
[      OK ] ResendTest.DeniesResendForSent (0 ms)
[ RUN    ] ResendTest.DeniesResendForCreated
[      OK ] ResendTest.DeniesResendForCreated (0 ms)
[ RUN    ] ResendTest.DeniesResendForSkipped
[      OK ] ResendTest.DeniesResendForSkipped (0 ms)
[-----] 4 tests from ResendTest (0 ms total)

[-----] Global test environment tear-down
[=====] 7 tests from 2 test suites ran. (0 ms total)
[ PASSED ] 7 tests.

```

Рисунок 3 — Результат запуска unit-тестов (7/7 PASSED)

7. Демонстрация доработанного функционала

7.1. Фича 1 — аналитика кампании

Запрос GET /campaigns/<id>/stats возвращает полную информацию о кампании. Ответ содержит поля: name, phone, message_template, recipient_name и sms_status — текущий статус отправки:

```
curl http://localhost:18080/campaigns/2/stats
```

7.2. Фича 2 — повторная отправка

Для проверки resend вручную выставлен статус failed через SQLite, после чего выполнена повторная отправка и проверка финального статуса:

```

# Устанавливаем статус failed
docker compose exec app sqlite3 /data/promo_notify.db \
    "UPDATE campaigns SET status='failed' WHERE id=2;"

# Повторная отправка
curl -X POST http://localhost:18080/campaigns/2/resend

```

```
# Проверяем что статус изменился на sent
curl http://localhost:18080/campaigns/2/stats
```

Результат выполнения команд представлен ниже. Статус кампании последовательно изменился: created → failed → sent.

```
whocares@DESKTOP-BT68DGB:~/promo-notify-cpp$ docker compose exec app \
  sqlite3 /data/promo_notify.db \
  "UPDATE campaigns SET status='failed' WHERE id=2;"

# Повторная отправка
curl -X POST http://localhost:18080/campaigns/2/resend
{"sent":true,"status":"ok"}

# Проверяем что статус изменился на sent
curl http://localhost:18080/campaigns/2/stats
{"recipient_name":"Михаил","message_template":"Привет, {name}! Скидка 30%",
  "phone":"79155596906","name":"Тест resend","sms_status":"sent",
  "id":2,"status":"ok"}
```


Выводы

В ходе лабораторной работы освоена методология Git Flow для управления разработкой программного обеспечения. Проект организован по схеме main / develop / feature-ветки. Для каждой из двух фич создавалась отдельная feature-ветка, разработка велась изолированно, после завершения ветка сливалась в develop через git flow feature finish.

Реализованы два нереализованных use case из ЛР №2: маршрут GET /campaigns/<id>/stats для получения аналитики по кампании и маршрут POST /campaigns/<id>/resend для повторной отправки SMS по кампаниям со статусом failed. Новый функционал покрыт семью unit-тестами с использованием Google Test — все тесты прошли успешно.

Применение Git Flow позволило структурировать историю изменений: граф коммитов наглядно отображает ветвление и слияние feature-веток. Написание unit-тестов подтвердило корректность реализованной бизнес-логики персонализации и повторной отправки.